INFORME INTEGRAL DE ACTIVIDADES

Machine Learning y Deep Learning

Compilación de Entregables: Actividades 1, 2, 3 y Clasificación de Pingüinos

Integrantes	Emanuel Zuluaga Restrepo Julián Canas Saldarriaga Nicolas Páez Lancheros
Profesor	Daniel Florez
Materia	Machine Learning y Deep Learning
Fecha	Abril de 2026

Contenido

1. Introducción General 4
2. Objetivos del Curso 5
  2.1 Objetivo General 5
  2.2 Objetivos Específicos 5
3. Entrega 1 — Manipulación de Imágenes con NumPy 6
  3.1 Descripción y Propósito 6
  3.2 Herramientas Utilizadas 6
  3.3 Ejercicios Desarrollados 6
    Ejercicio 1 — Gradiente Horizontal en Escala de Grises 6
    Ejercicio 2 — Patron Diagonal Alternado 6
    Ejercicio 3 — Crossfade entre Imágenes y GIF Animado 7
    Ejercicio 4 — Mosaico por Bloques con Filtrado de Canales 7
    Ejercicio 5 — Efecto Espejo Vertical 7
  3.4 Resultados y Verificación 7
  3.5 Aprendizajes y Conclusiones 8
4. Clasificación de Pingüinos (Palmer Penguins) 9
  4.1 Descripción y Propósito 9
  4.2 Dataset 9
  4.3 Análisis Exploratorio de Datos (EDA) 9
  4.4 Preparación de Datos 10
  4.5 Entrenamiento del Modelo 10
  4.6 Resultados y Evaluación 10
  4.7 Aprendizajes y Conclusiones 11
5. Entrega 2 — Comparación de Modelos de Clasificación sobre COVID-19 12
  5.1 Descripción y Propósito 12
  5.2 Dataset 12
  5.3 Entendimiento del Problema 12
  5.4 Análisis Exploratorio de Datos (EDA) 12
  5.5 Preparación de Datos 13
    Selección de Variables 13
    Estandarización de Texto 13
    Codificación 13
    Split y Escalado 13
  5.6 Modelos Implementados 13
    Regresión Lineal (Experimental) 13

Regresión Logística 14
K-Nearest Neighbors (KNN) 14
Naive Bayes (GaussianNB) 14
5.7 Experimentos Adicionales 15
5.8 Predicción sobre Caso Nuevo 15
5.9 Comparación Final y Conclusión 15
5.10 Aprendizajes y Conclusiones 15
6. Entrega 3 — Modelos No Supervisados y SVM sobre COVID-19 17
6.1 Descripción y Propósito 17
6.2 Dataset y Preparación 17
6.3 K-Means: Clustering como Feature Engineering 17
6.4 SVM con Kernel Lineal 18
6.5 SVM con Kernels No Lineales (Aproximación Nystroem) 18
6.6 Evaluación y Diagnostico de Overfitting 18
6.7 Ensamble por Votación (Hard Voting) 19
6.8 Stress Test con Clase Balanceada 19
6.9 Aprendizajes y Conclusiones 19
7. Metodología General del Curso 21
7.1 Entendimiento del Problema 21
7.2 Exploración y Análisis de Datos (EDA) 21
7.3 Preparación de Datos 21
7.4 Modelado y Entrenamiento 21
7.5 Evaluación y Comparación 21
8. Herramientas y Tecnologías Utilizadas 22
9. Conclusiones Generales 23
9.1 Sobre los Datos 23
9.2 Sobre los Modelos 23
9.3 Sobre la Evaluación 23
9.4 Sobre los Desafíos Computacionales 23

1. Introducción General

El presente documento es informe integral de todas las actividades desarrolladas durante el curso de Machine Learning y Deep Learning hasta el momento. A lo largo del semestre, el equipo trabajo en cuatro entregables que abarcan desde la manipulación de imágenes con operaciones matriciales hasta la implementación de modelos supervisados y no supervisados sobre conjuntos de datos reales de gran escala. Este informe recopila los objetivos, metodologías, resultados y aprendizajes de cada entrega, con el propósito de ofrecer una visión consolidada del proceso formativo y del nivel de competencia técnica alcanzado.

Los trabajos se organizaron de forma progresiva en complejidad: comenzando con fundamentos de representación de imágenes como matrices (Entrega 1), avanzando hacia la clasificación supervisada con un dataset clásico de biología (Clasificación de Pingüinos), profundizando en la comparación de múltiples modelos de clasificación sobre datos reales de salud pública (Entrega 2), y finalizando con la implementación de modelos no supervisados y kernels SVM en el mismo contexto (Entrega 3).

El dataset de referencia para los entregables 2 y 3 corresponde al registro oficial de casos positivos de COVID-19 en Colombia, suministrado por el Instituto Nacional de Salud (INS/SIVIGILA), con más de 1.5 millones de registros. Este escenario de alto volumen de datos exigió estrategias computacionales eficientes y permitió contrastar el comportamiento de los modelos en condiciones reales de producción.

2. Objetivos del Curso

2.1 Objetivo General

Desarrollar competencias técnicas y analíticas en Machine Learning mediante la implementación practica de modelos supervisados y no supervisados, aplicados a problemas reales de datos, con énfasis en el ciclo completo de un proyecto.

2.2 Objetivos Específicos

- Comprender la representación digital de imágenes como estructuras matriciales y aplicar transformaciones mediante operaciones sobre arreglos numéricos.
- Implementar flujos completos de Machine Learning: EDA, limpieza, ingeniería de características, entrenamiento y evaluación de modelos.
- Comparar el desempeño de múltiples algoritmos de clasificación (Regresion Logística, KNN, Naive Bayes, SVM, Arboles de Decisión) sobre el mismo problema y justificar la elección del modelo más adecuado.
- Aplicar técnicas de clustering (K-Means) como herramienta de feature engineering complementaria a la clasificación supervisada.
- Desarrollar criterio crítico para la interpretación de métricas de evaluación, especialmente en escenarios de desbalance de clases.
- Adquirir experiencia práctica con herramientas del ecosistema Python para ciencia de datos: NumPy, Pandas, Scikit-learn, Matplotlib y Seaborn.

3. Entrega 1 — Manipulación de Imágenes con NumPy

3.1 Descripción y Propósito

La primera entrega tuvo un carácter introductorio y pedagógico: el objetivo no era entrenar modelos predictivos, sino afianzar la comprensión fundamental de cómo se representan y manipulan las imágenes en el ámbito computacional. Partiendo del principio de que una imagen digital es esencialmente una matriz tridimensional de valores enteros (alto x ancho x canales RGB), los ejercicios propusieron generar y transformar imágenes utilizando únicamente operaciones de indexación y algebra lineal básica sobre arreglos de NumPy, sin recurrir a funciones especializadas de procesamiento de imágenes.

Esta restricción metodológica fue deliberada: al forzar al equipo a construir cada efecto desde cero, se consolidan los conceptos de espacio de color, canales RGB, indexación de pixeles y operaciones punto a punto, que son la base sobre la que se construyen todas las técnicas de visión por computador y deep learning.

3.2 Herramientas Utilizadas

- Python 3 como lenguaje principal.
- NumPy para creación y manipulación de arreglos matriciales.
- OpenCV (cv2) para guardar imágenes y convertir espacios de color (RGB a BGR).
- Matplotlib para visualización en el notebook.
- PIL (Pillow) para carga de imágenes externas.
- ImageIO para generación de GIF animado.
- Google Colab como entorno de ejecución.

3.3 Ejercicios Desarrollados

Ejercicio 1 — Gradiente Horizontal en Escala de Grises

Se creo una imagen sintética RGB de 300x350 pixeles. Para cada columna x de la imagen, se calculó un valor de intensidad proporcional a la posición horizontal, variando desde 0 (negro) hasta 255 (blanco). Dicho valor se asignó idénticamente a los tres canales (R, G, B) para producir una imagen en escala de grises con degradado suave de izquierda a derecha. El resultado evidencia como la intensidad de un pixel puede controlarse matemáticamente mediante su índice de columna.

Ejercicio 2 — Patron Diagonal Alternado

Se creo una imagen RGB de 512x512 pixeles. El recorrido por todos los pixeles (y, x) aplico una condición matemática basada en la diferencia (x - y) dividida en bloques de 26 unidades: según

el residuo de esta división modulo 2, el pixel tomaba el valor blanco o negro. El resultado es un patrón diagonal de franjas alternadas, similar a una tela a cuadros rotada 45 grados. Este ejercicio enfatiza el uso de aritmética de índices para definir regiones visuales sin usar funciones de dibujo.

Ejercicio 3 — Crossfade entre Imágenes y GIF Animado

Utilizando las imágenes clásicas de 'Lena' y 'Bengal Tiger', ambas redimensionadas a 512x512 pixeles, se generaron 30 frames intermedios aplicando una mezcla lineal alfa entre las dos imágenes: el frame i es (1-alfa)*lena + alfa*tigre, donde alfa varia de 0 a 1. Los 30 frames se exportaron como GIF animado con duración de 0.08 segundos por frame, produciendo una transición visual fluida. Este ejercicio introduce el concepto de interpolación entre tensores de imágenes, fundamental en técnicas de aumento de datos (data augmentation) y en modelos generativos.

Ejercicio 4 — Mosaico por Bloques con Filtrado de Canales

Sobre la imagen Lena redimensionada a 640x480, se definió un tamaño de bloque de 80 pixeles. Recorriendo la imagen por bloques, se asignó a cada región uno de tres 'tipos' según el índice del bloque modulo 3: el tipo 0 conserva solo el canal rojo (anulando verde y azul), el tipo 1 conserva solo el verde, y el tipo 2 solo el azul. El resultado es un mosaico donde cada cuadricula se tiñe de un color primario diferente. El ejercicio refuerza la comprensión de la estructura tridimensional RGB y la manipulación selectiva de canales.

Ejercicio 5 — Efecto Espejo Vertical

Se cargo la imagen Lena original. Se extrajo la mitad superior (primeras h/2 filas) y se generó su reflejo vertical usando np.flipud(). Ambas mitades se concatenaron verticalmente con np.vstack, produciendo una imagen donde la mitad inferior es el espejo exacto de la superior. Esta transformación, conceptualmente simple, sirve como base para técnicas de aumento de datos y preprocesamiento en redes neuronales convolucionales.

3.4 Resultados y Verificación

Todos los ejercicios se ejecutaron exitosamente en Google Colab. Las llamadas a cv2.imwrite() retornaron True en todos los casos, confirmando la escritura exitosa de los archivos. Las imágenes y el GIF generados fueron visualizados dentro del notebook y exportados como evidencia. El flujo completo (generación matricial, visualización y guardado) funciono de principio a fin sin errores.

3.5 Aprendizajes y Conclusiones

Esta entrega establece un fundamento conceptual critico: toda imagen procesada por un modelo de deep learning o visión por computador es, en última instancia, una estructura numérica sobre la cual se aplican operaciones matemáticas. Al construir efectos visuales desde cero usando solo indexación y algebra, el equipo interiorizo la naturaleza matricial de los datos de imagen, la estructura de canales RGB, el impacto de la indexación de pixeles en los patrones visuales, la interpolación lineal como primitiva de mezcla, y la diferencia entre espacios de color RGB (Matplotlib) y BGR (OpenCV).

4. Clasificación de Pingüinos (Palmer Penguins)

4.1 Descripción y Propósito

Esta actividad introdujo el flujo completo de un proyecto de Machine Learning supervisado, desde la exploración del dataset hasta la evaluación del modelo entrenado. Se utilizo el dataset Palmer Penguins, un conjunto de datos de referencia en la comunidad de ciencia de datos que contiene mediciones físicas de tres especies de pingüinos (Adelie, Chinstrap y Gentoo) capturadas en el archipiélago Palmer, en la Antártida. El objetivo fue clasificar la especie de cada individuo a partir de sus características morfológicas y contextuales.

4.2 Dataset

Archivo	penguins.csv
Total de registros	344 filas, 9 columnas
Variable objetivo	especies (Adelie, Chinstrap, Gentoo)
Variables numéricas	bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, year
Variables categóricas	island, sex
Columna descartada	Unnamed: 0 (índice redundante sin valor analítico)
Registros tras limpieza	333 (se eliminaron 11 filas con valores nulos)

4.3 Análisis Exploratorio de Datos (EDA)

El proceso de exploración comenzó con la revisión de la estructura del dataset usando df.info() y df.head(), identificando los tipos de datos y la presencia de valores faltantes. Las principales variables numéricas presentaban 2 nulos cada una, mientras que la variable sex acumulaba 11 valores faltantes.

Se aplicaron estadísticas descriptivas (df.describe()) para caracterizar la distribución de cada variable, y se utilizó value_counts() sobre la variable objetivo para cuantificar el balance de clases. El análisis revelo un desbalance moderado entre las tres especies, con Adelie como la clase más frecuente.

Para el análisis de relaciones, se generó un pairplot con hue por especie, que permitió identificar los patrones de separación entre clases. Los hallazgos más relevantes fueron:

- Correlación positiva fuerte entre flipper_length_mm y body_mass_g.
- Separación visual clara entre especies en las variables flipper_length_mm y body_mass_g.

- Solapamiento moderado en bill_length_mm y bill_depth_mm, especialmente entre Chinstrap y Adelie.
- Ausencia de outliers extremos en las distribuciones univariadas.
- Multimodalidad en los histogramas de varias variables, producto de la mezcla de especies.

4.4 Preparación de Datos

Las acciones de preparación realizadas fueron:

- Eliminación de la columna Unnamed: 0 por carecer de valor analítico.
- Eliminación de filas con valores nulos (dropna): se pasó de 344 a 333 registros. Se eligió eliminar en lugar de imputar dado el bajo porcentaje de nulos (~3%) y el riesgo de sesgar el modelo con imputaciones inadecuadas.
- One-Hot Encoding (pd.get_dummies con drop_first=True) para las variables island y sex, eliminando la columna redundante para evitar multicolinealidad.
- Label Encoding con LabelEncoder sobre la variable objetivo species para convertirla a valores numericos.

Las columnas finales del dataset modelado fueron: bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, year, island_Dream, island_Torgersen, sex_male.

4.5 Entrenamiento del Modelo

Se separo el dataset en 80% entrenamiento (266 registros) y 20% prueba (67 registros), usando stratify=y para mantener las proporciones de clase y random_state=42 para reproducibilidad.

El modelo elegido fue un Árbol de Decisión (DecisionTreeClassifier) con max_depth=3 y random_state=42. La restricción de profundidad máxima fue una decisión deliberada para controlar la complejidad del modelo y reducir el riesgo de sobreajuste, a la vez que mantiene la interpretabilidad (el árbol hace preguntas binarias tipo si/no sobre umbrales de variables).

4.6 Resultados y Evaluación

El modelo se evaluó sobre el conjunto de prueba (67 muestras). Los resultados del reporte de clasificación fueron:

Clase	Precisión	Recall	F1-Score	Soporte
Adelie	0.91	1.00	0.95	29 muestras
Chinstrap	1.00	0.93	0.96	14 muestras
Gentoo	1.00	0.92	0.96	24 muestras
Accuracy global	-	-	0.96	67 muestras
Macro avg F1	-	-	0.96	67 muestras

El modelo logro un desempeño solido con accuracy de 0.96. El recall perfecto en la clase Adelie indica que prácticamente no se dejaron ejemplares de esta especie sin detectar. La matriz de confusión no reporto errores graves; las confusiones menores tendieron a concentrarse entre las clases con mayor solapamiento morfológico en las variables del pico.

4.7 Aprendizajes y Conclusiones

Esta actividad consolido el flujo completo de Machine Learning supervisado. Los aprendizajes principales incluyen: la importancia del EDA antes de modelar, el impacto del desbalance de clases en la elección de métricas (F1-Score vs accuracy), la necesidad de codificar variables categóricas antes de entrenar, el efecto de la profundidad del árbol en la complejidad del modelo, y el uso del split estratificado para garantizar representatividad en el conjunto de prueba. Adicionalmente, se identificó que flipper_length_mm y body_mass_g son las variables con mayor poder discriminativo para separar las tres especies.

5. Entrega 2 — Comparación de Modelos de Clasificación sobre COVID-19

5.1 Descripción y Propósito

El segundo entregable escalo significativamente en complejidad: se abandonó el dataset de juguete para trabajar con un conjunto de datos de salud pública real, con más de 1.5 millones de registros. El objetivo fue predecir el desenlace clínico final de un caso positivo de COVID-19 en Colombia (recuperación vs fallecimiento) comparando cuatro familias de modelos: Regresión Lineal, Regresion Logística, K-Nearest Neighbors y Naive Bayes.

5.2 Dataset

Fuente	INS / SIVIGILA — Casos positivos COVID-19 Colombia
Total de filas	1,597,743
Total de columnas	23
Variable objetivo	Recuperado (RECUPERADO=0, FALLECIDO=1)
Ruta del archivo	/content/drive/MyDrive/casos_covid_19.csv

5.3 Entendimiento del Problema

Se definió el problema como clasificación binaria: dado un conjunto de variables inicialmente disponibles al momento del diagnóstico (edad, sexo, ubicación, tipo de contagio, etnia, estado clínico), predecir si el paciente sobrevivirá o fallecerá. Este problema es de alta relevancia porque permite:

- Implementar sistemas de triaje inteligente que prioricen vigilancia para pacientes de alto riesgo.
- Anticipar la demanda de recursos hospitalarios, particularmente camas UCI.
- Identificar zonas geográficas y grupos demográficos con mayor vulnerabilidad.

5.4 Análisis Exploratorio de Datos (EDA)

El EDA sobre el dataset de COVID-19 presento desafíos adicionales respecto al dataset de pingüinos, derivados del volumen y la calidad del registro administrativo. Los hallazgos más relevantes fueron:

- La variable Edad resulto ser el predictor más fuerte, con alta correlación con el desenlace clínico.
- Se detecto ruido por inconsistencias de capitalización en columnas de texto (por ejemplo, 'F' vs 'f' en Sexo), lo que exigió estandarización previa al modelado.

- Varias columnas presentaban un alto número de valores nulos por naturaleza del registro: fechas de muerte en pacientes recuperados, información de país de procedencia y grupo étnico.
- La variable objetivo Recuperado estaba completa (sin nulos en la etiqueta), lo que facilito el proceso de entrenamiento.

5.5 Preparación de Datos

El pipeline de preparación incluyo los siguientes pasos:

Selección de Variables

Se seleccionaron siete columnas como predictores: Edad, Sexo, Tipo de contagio, Nombre departamento, Pertenencia étnica, Estado, y la variable objetivo Recuperado.

Estandarización de Texto

Se aplico conversión a mayúsculas (.str.upper()) y eliminación de espacios (.str.strip()) a las columnas de texto para eliminar duplicidades categóricas por mayúsculas/minusculas. Los registros con valores inconsistentes o nulos en Estado fueron eliminados mediante dropna().

Codificación

La variable objetivo Recuperado se binarizo manualmente (RECUPERADO=0, FALLECIDO=1). Las variables categoricas (Sexo, Tipo de contagio, Estado, Nombre departamento) se transformaron con pd.get_dummies(drop_first=True) para producir variables indicadoras binarias.

Split y Escalado

Se realizo un train/test split 80/20 con stratify=y y random_state=42. Se aplico StandardScaler para escalar las variables, operación especialmente critica para Edad (rango 0-114) frente a las variables dummies (rango 0-1), que de otro modo habrían sesgado los modelos basados en distancia como KNN.

5.6 Modelos Implementados

Regresión Lineal (Experimental)

Se entreno un LinearRegression() sobre los datos escalados, convirtiendo la predicción continua a clase binaria mediante un umbral de 0.5. Esta implementación fue deliberadamente experimental: su propósito fue demostrar por que la regresión lineal no es el enfoque correcto para clasificación binaria, ya que puede producir valores de probabilidad fuera del rango [0,1] y no genera fronteras de decisión óptimas para este tipo de problema.

Accuracy	0.9863

MSE	0.0773
RMSE	0.2779

Regresión Logística

Se entreno un LogisticRegression(max_iter=1000, solver='lbfgs') sobre el conjunto completo de entrenamiento escalado. Este modelo aplica la función sigmoide para producir probabilidades coherentes entre 0 y 1, lo que lo convierte en el candidato conceptualmente correcto para clasificación binaria. Resulto ser el modelo con mejor desempeño global, combinando alta precisión, eficiencia computacional y escalabilidad.

Accuracy	0.9982
F1-Score (Macro)	~0.9968

K-Nearest Neighbors (KNN)

Dado el tamaño masivo del dataset (1.5M registros), entrenar KNN sobre el conjunto completo era computacionalmente inviable. Se opto por una muestra estratificada del 5% del conjunto de entrenamiento para el ajuste del modelo. Se uso KNeighborsClassifier(n_neighbors=5, weights='distance', algorithm='auto', n_jobs=-1).

Accuracy (sobre muestra)	0.9968
F1-Score (sobre muestra)	0.9942

La ponderación por distancia (weights='distance') asigna mayor peso a los vecinos más cercanos, lo que mejora el desempeño en fronteras de decisión difusas. A pesar de su buen rendimiento en la muestra, el modelo es prohibitivo en tiempo de inferencia para el dataset completo.

Naive Bayes (GaussianNB)

Se entreno un GaussianNB() sobre el conjunto completo escalado. Aunque su accuracy global fue 0.726, el análisis por clase revelo el problema fundamental del modelo: recall extremadamente bajo para la clase Fallecido (~0.03) a pesar de un AUC de 0.9419. Esto significa que el modelo casi nunca identifica un caso de fallecimiento, lo cual es inaceptable desde una perspectiva clínica.

Accuracy	0.7260
AUC-ROC	0.9419
Recall Fallecido	~0.03

El problema de Naive Bayes reside en su supuesto de independencia condicional entre variables, que raramente se cumple en datos médicos reales. Aunque el AUC sugiere buena capacidad de separación por score, el umbral por defecto (0.5) no captura adecuadamente la clase minoritaria.

5.7 Experimentos Adicionales

Se realizaron dos experimentos para complementar la comparación:

- Experimento 1 (KNN con variaciones): se probó KNN con diferentes valores de k (3, 7, 15), con test_size del 30%, eliminando la variable Edad. El experimento demostró la sensibilidad del modelo al hiperparametro k y la importancia de Edad como predictor dominante.
- Experimento 2 (Estabilidad por semilla): se repitió el split con semillas 10, 42 y 100 para evaluar la estabilidad del modelo. La variación máxima de accuracy fue ~0.002, confirmando que el modelo es estable ante distintos seeds aleatorios.

5.8 Predicción sobre Caso Nuevo

Se selecciono como caso de prueba el paciente de mayor edad en el conjunto de test del KNN. Las probabilidades de fallecimiento predichas por los modelos fueron: KNN pronostico un 83.1% de probabilidad de fallecimiento. Naive Bayes produjo una probabilidad extrema cercana al 100%, evidenciando el problema de exceso de confianza que genera este modelo cuando sus supuestos de independencia no se cumplen.

5.9 Comparación Final y Conclusión

Modelo	Accuracy	F1-Score	Escalabilidad	Recomendación
Regresión Logística	0.9982	0.9968	Alta	Recomendado
KNN (muestra)	0.9968	0.9942	Baja	Uso individual
Regresión Lineal	0.9863	N/A	Alta	No aplica
Naive Bayes	0.7260	0.0641	Alta	No recomendado

La Regresión Logística fue seleccionada como modelo recomendado por su combinación de alto desempeño, eficiencia computacional, escalabilidad con 1.5M de registros y correcta fundamentación teórica para clasificacion binaria.

5.10 Aprendizajes y Conclusiones

Esta entrega fue la más rica en aprendizajes metodológicos. Se comprendio en profundidad por que la regresion lineal es conceptualmente inadecuada para clasificación, se experimentó el costo computacional de KNN con datasets masivos, y se identificó el riesgo de Naive Bayes en

datos con correlaciones entre variables. El trabajo sobre un dataset real de 1.5M registros también exigió desarrollar criterio sobre muestreo estratificado, eficiencia de memoria y uso de paralelismo (n_jobs=-1). La distinción entre accuracy y F1-score como métricas de evaluación se volvió concreta y relevante al enfrentar el desbalance de clases.

6. Entrega 3 — Modelos No Supervisados y SVM sobre COVID-19

6.1 Descripción y Propósito

La tercera entrega extendió el trabajo sobre el dataset COVID-19 en dos dimensiones: se incorporaron técnicas de aprendizaje no supervisado (K-Means) para descubrir estructura latente en los datos, y se implementaron modelos SVM con distintos kernels para la clasificación supervisada. Adicionalmente, se construyó un ensamble por votación y se realizó un stress test orientado a minimizar falsos negativos en la detección de fallecimientos.

La motivación detrás de combinar clustering con clasificación supervisada responde a una estrategia de feature engineering: los clusters generados por K-Means capturan 'perfiles de paciente' en el espacio de características, y esa información de agrupamiento se agrega como una nueva variable predictor a los modelos SVM, enriqueciendo su representación del problema.

6.2 Dataset y Preparación

Se utilizo el mismo dataset de la Entrega 2 (1,597,743 registros, 23 columnas). La preparación incorporo mejoras respecto a la entrega anterior:

- Estandarización de texto (conversión a mayúsculas) para columnas categoricas.
- Feature engineering sobre la variable Edad: normalización según la unidad de medida (anos, meses o días) para evitar que el modelo confunda 5 meses con 5 años.
- Creación de la variable objetivo binaria Target (1=Fallecido, 0=Recuperado).
- One-Hot Encoding sobre Sexo y Tipo de contagio.
- Manejo de valores nulos mediante imputación con la mediana (X.fillna(X.median())).
- StandardScaler para escalar todas las variables (esencial para SVM y K-Means).
- Train/test split 80/20 con stratify=y y random_state=42.

6.3 K-Means: Clustering como Feature Engineering

Para determinar el número optimo de clusters, se implementó el método del codo: se calculó la inercia del modelo para valores de k entre 1 y 10 usando MiniBatchKMeans (algoritmo optimizado para datasets masivos que procesa lotes de datos en lugar del dataset completo). La grafica de inercia vs k mostro un punto de inflexión que llevo a seleccionar k=5 como numero optimo de clusters.

El modelo final MiniBatchKMeans(n_clusters=5, batch_size=10000, n_init=3) fue entrenado sobre el conjunto de entrenamiento escalado. El identificador de clúster predicho para cada muestra fue agregado como una columna adicional en X_train y X_test, enriqueciendo la representación de entrada para los modelos SVM posteriores.

Conceptualmente, los 5 clusters pueden interpretarse como 'perfiles demografico-clinicos' de pacientes: grupos de individuos con características similares de edad, ubicación, tipo de contagio y sexo. Esta información de contexto grupal ayuda a los clasificadores a distinguir mejor entre perfiles de alto y bajo riesgo.

6.4 SVM con Kernel Lineal

Para escalar SVM a 1.5M de registros, se usó LinearSVC(dual=False, max_iter=2000) envuelto con CalibratedClassifierCV(cv=3) para obtener probabilidades calibradas y poder evaluar la curva ROC. LinearSVC optimiza la función de perdida hinge directamente en el espacio primal, lo que es mucho más eficiente que la formulación dual para datasets grandes.

6.5 SVM con Kernels No Lineales (Aproximación Nystroem)

La aplicación directa de kernels no lineales (RBF, polinomial) sobre 1.5M de registros es computacionalmente inviable por el costo O(n^2) o O(n^3) de calcular la matriz de Gram completa. Para superar esta limitación, se usó la aproximación Nystroem: una técnica que aproxima el mapa de características del kernel usando un subconjunto de 150 componentes, permitiendo aplicar un LinearSVC sobre la representación aproximada.

- Kernel RBF (Radial Basis Function): Nystroem(kernel='rbf', gamma=0.1, n_components=150) + LinearSVC.
- Kernel Polinomial: Nystroem(kernel='poly', degree=3, n_components=150) + LinearSVC.

Se uso decision_function como score para evaluar ROC en lugar de predict_proba, reduciendo el consumo de memoria. Se aplico liberación explicita de memoria (gc.collect()) después de cada modelo para evitar saturar la RAM de Colab.

6.6 Evaluación y Diagnostico de Overfitting

Se realizo una comparación sistemática de métricas entre el conjunto de entrenamiento y el conjunto de prueba para diagnosticar overfitting o underfitting:

Accuracy (Train)	0.998005
Accuracy (Test)	0.998069
F1-Score (Train)	0.996425
F1-Score (Test)	0.996539

Las métricas son prácticamente idénticas entre entrenamiento y prueba, lo que indica que el modelo no presenta overfitting significativo y generaliza bien a datos no vistos. Este resultado valida la robustez del pipeline de preparación y la capacidad del modelo lineal para capturar los patrones relevantes del problema.

6.7 Ensamble por Votación (Hard Voting)

Se construyo un ensamble combinando las predicciones de los tres modelos SVM entrenados (lineal, RBF aproximado, polinomial aproximado). La regla de decisión fue mayoritaria: una muestra se clasifica como Fallecido si al menos 2 de los 3 modelos la predicen como tal. Los resultados del ensamble fueron:

Accuracy del ensamble	0.9998
Precisión (ambas clases)	~1.00
Recall (ambas clases)	~1.00

El ensamble por votación logro el mejor desempeño global de toda la entrega, superando a cualquier modelo individual. La diversidad entre kernels permitió compensar los errores individuales de cada modelo.

6.8 Stress Test con Clase Balanceada

Desde una perspectiva clínica, el error más costoso es un falso negativo: predecir que un paciente se recuperará cuando en realidad fallecerá. Por esto, se realizó un stress test entrenando un LinearSVC(class_weight='balanced'), que ajusta internamente los pesos de clase inversamente proporcional a su frecuencia, penalizando más los errores en la clase minoritaria (Fallecido, 27.81% del total).

Accuracy global	0.9677
Precisión (Fallecidos)	0.8961
Recall (Fallecidos)	0.9998
F1-Score (Fallecidos)	0.9451
ROC-AUC	0.9974

Al balancear las clases, se logró un recall casi perfecto para Fallecidos (0.9998): el modelo detecta prácticamente todos los casos de fallecimiento. El costo de esta mejora es una reducción en accuracy global (de 0.9998 a 0.9677), ya que algunos recuperados son ahora clasificados como fallecidos (falsos positivos). En un contexto clínico, este intercambio es aceptable e incluso deseable, pues es preferible ser conservador en la predicción de fallecimiento.

6.9 Aprendizajes y Conclusiones

La tercera entrega consolido una comprensión madura de los modelos de clasificación avanzada. Los aprendizajes más significativos incluyen: el uso de K-Means como herramienta de feature engineering (no solo descriptiva), las limitaciones computacionales de SVM con kernels exactos

y la elegancia de la aproximación Nystroem para superarlas, la diferencia practica entre accuracy y recall en problemas desbalanceados clínicamente relevantes, la ventaja del ensamble por votación para reducir la varianza del error, y el concepto de class_weight='balanced' para ajustar la sensibilidad del clasificador según los costos relativos de cada tipo de error.

7. Metodología General del Curso

A lo largo de los cuatro entregables, el equipo siguió una metodología de trabajo consistente y progresiva que puede sintetizarse en las siguientes etapas:

7.1 Entendimiento del Problema

Cada actividad comenzó con la definición clara del tipo de tarea (clasificación, regresion, clustering, procesamiento de imágenes) y de las variables de interés. Para los problemas de salud pública, se estableció explícitamente la relevancia clínica del problema y los tipos de error más costosos, lo cual guio las decisiones de modelado posteriores.

7.2 Exploración y Análisis de Datos (EDA)

El EDA fue sistemático en todos los entregables: revisión de estructura con df.info(), estadísticas descriptivas con df.describe(), análisis de distribuciones con histogramas y boxplots, análisis de relaciones con pairplots y matrices de correlación, y cuantificación de valores faltantes. Para el dataset COVID-19, se agregó el análisis de calidad de texto (ruido por mayúsculas/minúsculas) como paso previo.

7.3 Preparación de Datos

El pipeline de preparación evoluciono en complejidad a lo largo del semestre: desde la simple eliminación de columnas redundantes y filas nulas (pingüinos), hasta pipelines con estandarización de texto, normalización de unidades de medida, codificación múltiple y escalado (COVID-19). En todos los casos se priorizo la reproducibilidad mediante random_state fijo y la prevención de data leakage mediante fit/transform correcto en train vs test.

7.4 Modelado y Entrenamiento

Se implementaron modelos de creciente sofisticación: Árbol de Decisión, Regresion Lineal y Logística, KNN, Naive Bayes, SVM lineal, SVM con kernels no lineales (vía Nystroem) y ensambles. En todos los casos se documentó la fundamentación teórica del modelo y sus supuestos, para poder interpretar correctamente sus limitaciones en cada contexto.

7.5 Evaluación y Comparación

La evaluación fue siempre multi-metrica: accuracy, precisión, recall, F1-score por clase, AUC-ROC cuando fue pertinente, y diagnóstico de overfitting mediante comparación train/test. Se utilizo la matriz de confusión como herramienta visual para identificar patrones de error específicos. En los entregables 2 y 3 se realizó además una tabla comparativa de modelos para facilitar la decisión final.

8. Herramientas y Tecnologías Utilizadas

El ecosistema tecnológico del curso fue consistente a lo largo de todos los entregables, centrado en Python y sus librerías científicas:

Herramienta / Librería	Uso Principal
Python 3	Lenguaje de programación principal
NumPy	Manipulación matricial, operaciones sobre arreglos de imágenes
Pandas	Carga, manipulación y transformación de datos tabulares
Scikit-learn	Modelos ML, preprocesamiento, métricas de evaluación
Matplotlib	Visualización básica, histogramas, matrices de confusión
Seaborn	Visualización estadística avanzada (pairplot, heatmaps, KDE)
OpenCV (cv2)	Guardado y conversión de espacios de color en imágenes
PIL (Pillow)	Carga de imágenes externas
ImageIO	Generación de GIF animado
Google Colab	Entorno de ejecución en la nube con GPU/RAM compartida
Google Drive	Almacenamiento del dataset grande (casos COVID-19)

9. Conclusiones Generales

9.1 Sobre los Datos

La experiencia de trabajar con datasets de diferente naturaleza (matrices de imágenes, datos tabulares pequeños con el dataset Palmer Penguins, y un corpus real de 1.5 millones de registros) permitió desarrollar criterio sobre como adaptar el pipeline a las características del problema. Se aprendió que la calidad del preprocesamiento determina, en gran medida, el techo de desempeño del modelo.

9.2 Sobre los Modelos

La comparación sistemática de múltiples algoritmos sobre el mismo problema demostró que no existe un modelo universalmente superior: la elección correcta depende del tamaño del dataset, el tipo de frontera de decisión, los recursos computacionales disponibles y el costo relativo de cada tipo de error. La Regresion Logística demostró ser una opción robusta, eficiente y escalable para clasificación binaria con datos tabulares de gran volumen. El SVM con ensamble alcanzo las mejores métricas absolutas, mientras que el stress test con class_weight='balanced' mostro como adaptar el modelo a los requerimientos clínicos del problema.

9.3 Sobre la Evaluación

Uno de los aprendizajes más importantes del curso fue desarrollar criterio crítico para la interpretación de métricas. El accuracy solo es una métrica adecuada cuando las clases están balanceadas. En presencia de desbalance, el F1-score, el recall por clase y el AUC-ROC son mucho más informativos. Esta comprensión es fundamental para cualquier aplicación real de Machine Learning en contextos como salud, finanzas o seguridad.

9.4 Sobre los Desafíos Computacionales

El trabajo con 1.5 millones de registros introdujo restricciones reales de cómputo que obligaron al equipo a tomar decisiones pragmáticas: muestreo estratificado para KNN, uso de MiniBatchKMeans para clustering, aproximación Nystroem para kernels SVM no lineales y gestión explicita de memoria. Estas decisiones reflejan las habilidades que se requieren en entornos de producción reales.